

UNIVERSIDAD AUTÓNOMA DE BUCARAMANGA

Facultad de Ingeniería
Asignatura: Arquitectura de Software

INFORME TÉCNICO DE DISEÑO DE MICROSERVICIOS

FASE 1 DE 2

Plataforma Web Inteligente de Pair Programming Asistido por IA

Docentes: Mg. Fabián Suárez / Mg. Javier Pinzón
*Bibliografía base: Pacheco, V. F. (2018). Microservice Patterns
and Best Practices. Packt.*

Juan sebastian Jimenez Daza

George Sebastian Makhlouf Ruiz

Bucaramanga, Colombia — Mayo de 2026

1. Introducción

La Universidad Autónoma de Bucaramanga (UNAB) requiere una plataforma web colaborativa de Pair Programming asistida por Inteligencia Artificial para fortalecer la enseñanza de programación. El presente informe técnico describe el diseño conceptual de la arquitectura de microservicios, tomando como referencia principal el libro *Microservice Patterns and Best Practices* de Vinicius Feitosa Pacheco (2018, Packt), bibliografía base indicada por el docente.

La plataforma es una aplicación web (Single Page Application — SPA) construida con React.js, accesible desde cualquier navegador moderno, sin necesidad de instalación. El modelo de Pair Programming establece dos roles: driver (quien escribe el código) y navigator (quien revisa y guía), potenciados por sugerencias de IA en tiempo real.

1.1 Objetivos del diseño

- Diseñar una arquitectura de microservicios escalable y desacoplada para una plataforma web, siguiendo los patrones de Pacheco (2018).
- Implementar colaboración en tiempo real mediante WebSockets y algoritmos OT/CRDT en el editor de código web (Monaco Editor o CodeMirror).
- Desacoplar el motor de IA educativa usando Apache Kafka como bus de eventos asíncronos.
- Aplicar CQRS para optimizar los dashboards académicos del docente en el navegador.
- Garantizar resiliencia mediante Circuit Breaker, Chaos Engineering y despliegues Blue-Green/Canary.

1.2 Alcance

Este informe cubre la Fase 1: diseño conceptual de microservicios, arquitectura cliente web-microservicio, pipeline CI/CD con las etapas de pruebas de Pacheco (Cap. 12 y 13), y la Prueba del Mono con las siete categorías del Simian Army. La Fase 2 contemplará la implementación técnica con contratos OpenAPI 3.0 y scripts de Kubernetes.

2. Revisión de la Solución con Microservicios

2.1 Justificación de la arquitectura (Pacheco, Cap. 1 y 4)

Según Pacheco, los microservicios permiten independencia de despliegue, escalabilidad selectiva y aislamiento de fallos. Cada servicio tiene su propia base de datos (patrón Database per Service), puede escalar y desplegarse sin afectar a los demás, y puede implementarse en el lenguaje más adecuado para su dominio (polyglot). Para una plataforma web educativa, esto es fundamental porque los picos de uso del editor colaborativo y del motor de IA son completamente distintos.

- Escalabilidad independiente: ms-editor escala cuando muchas parejas editan simultáneamente; ms-ai escala según la demanda de sugerencias de código.

- Desacoplamiento tecnológico: ms-ai usa Python/FastAPI para el modelo LLM; ms-editor usa Node.js para WebSockets; ms-reports usa Java/Spring Boot para CQRS.
- Resiliencia: si ms-ai falla, el editor web continúa operando sin asistencia inteligente gracias al Circuit Breaker.
- Equipos independientes: cada microservicio puede desarrollarse y desplegarse de forma autónoma por equipos distintos.

2.2 Justificaciones técnicas requeridas por el enunciado

¿Cómo se sincroniza el editor colaborativo web?

El ms-editor implementa Operational Transformation (OT) o CRDT para resolver conflictos de edición concurrente. En el navegador, el estudiante usa Monaco Editor (el mismo editor de VS Code, adaptado para web) o CodeMirror. Cada keystroke genera una operación enviada vía WebSocket al ms-editor. El servidor aplica la transformación, resuelve conflictos y retransmite la operación transformada a todos los participantes de la sesión. Redis Pub/Sub actúa como broker entre instancias del ms-editor (Pacheco, Cap. 2). El estado del documento se persiste en MongoDB para recuperación ante desconexiones.

¿Cómo se desacoplan los servicios de IA? (Pacheco, Cap. 10 — Asynchronous Messaging)

El ms-ai opera de forma completamente asíncrona mediante Apache Kafka. El ms-editor publica eventos 'code_snapshot' en un topic de Kafka cada 10 segundos. El ms-ai los consume, procesa el código con el modelo LLM y publica sugerencias en el topic 'ai-suggestions'. El ms-editor consume las sugerencias y las muestra en el panel lateral del editor web sin bloquear la edición. Este desacoplamiento garantiza que la IA nunca bloquea la experiencia del usuario en el navegador.

¿Qué ocurre si falla el servicio de recomendaciones? (Pacheco, Cap. 4 — Circuit Breaker)

El sistema implementa el patrón Circuit Breaker entre ms-editor y ms-ai:

- Estado Closed: ms-ai responde correctamente, las sugerencias se muestran en el panel web del editor.
- Estado Open: tras 5 fallos consecutivos en 30 segundos, el circuito se abre. Las peticiones al ms-ai se bloquean sin esperar timeout. El navegador muestra un aviso visual: 'Asistencia IA temporalmente no disponible'.
- Estado Half-Open: tras 60 segundos, se envían peticiones de prueba para verificar recuperación.
- Fallback: con el circuito abierto, el editor muestra snippets locales precargados en el navegador.

¿Cómo se procesan eventos colaborativos en tiempo real? (Pacheco, Cap. 2 — Kafka)

Apache Kafka actúa como columna vertebral del procesamiento de eventos. Los topics principales son: 'session-events' (inicio/fin de sesión web, cambio de roles

driver/navigator), 'code-events' (snapshots del código del editor web), 'ai-suggestions' (respuestas del motor LLM), 'evaluation-completed' (resultados de ejecución en sandbox) y 'collaboration-metrics' (datos de participación para el ms-metrics e InfluxDB).

¿Qué patrones de microservicios son adecuados para aplicaciones colaborativas web? (Pacheco, Caps. 2–13)

- API Gateway (Pacheco, Cap. 13): Kong como punto de entrada único para todos los clientes web. Valida JWT, aplica rate limiting y enruta: /auth→ms-auth, /editor→ms-editor, /ai→ms-ai, /reports→ms-reports.
- SSO — Single Sign-On (Pacheco, Cap. 13): OAuth2 + OpenID Connect. Los usuarios se autentican una vez con credenciales UNAB. Token JWT (RS256, 15 min) almacenado en memoria del SPA; refresh token en cookie HTTPOnly.
- Circuit Breaker (Pacheco, Cap. 4): resiliencia en la comunicación ms-editor → ms-ai. Degradación elegante en el navegador.
- Event Sourcing (Pacheco, Cap. 3): ms-editor almacena todas las operaciones OT como eventos inmutables en MongoDB, permitiendo reconstruir el estado del documento en cualquier punto del tiempo.
- CQRS (Pacheco, Cap. 3 y 6): ms-reports separa QueryStack (lectura para dashboards web del docente) de CommandStack (escritura de métricas). Si el CommandStack falla, el dashboard sigue operando (Pacheco, Cap. 12).
- Database per Service (Pacheco, Cap. 4): cada microservicio tiene su propia base de datos independiente.
- Sidecar Pattern: Prometheus exporter en cada MS para observabilidad sin modificar el código de negocio.
- Asynchronous Messaging (Pacheco, Cap. 10): comunicación asíncrona ms-editor → ms-ai vía Kafka desacopla la IA del editor web.

3. Diagrama de Bloques de Microservicios

La plataforma se compone de nueve (9) microservicios organizados en cinco capas: Clientes Web (React.js SPA), API Gateway, Microservicios de Negocio, Bus de Eventos Kafka e Infraestructura de Datos. Cada microservicio tiene su propia base de datos siguiendo el patrón Database per Service (Pacheco, Cap. 4).

La Capa 1 contempla únicamente clientes web (navegador), ya que la plataforma es una aplicación web accesible desde cualquier dispositivo con navegador moderno, sin aplicaciones nativas ni plugins de IDE.

Microservicio	Dominio	Responsabilidad principal	Base de datos	Protocolo
ms-auth	Usuarios y autenticación	SSO · OAuth2 · JWT RS256 · Roles (estudiante/docente/admin)	PostgreSQL	REST

Microservicio	Dominio	Responsabilidad principal	Base de datos	Protocolo
ms-pairing	Pair Programming web	Parejas · driver/navigator · Sesiones colaborativas	PostgreSQL	REST / Events
ms-editor	Editor colaborativo web	OT/CRDT · Monaco/CodeMirror · WebSocket · Event Sourcing	MongoDB + Redis	WebSocket
ms-ai	Motor IA educativa	LLM · Sugerencias async · Desacoplado vía Kafka	MongoDB	Kafka (async)
ms-evaluation	Evaluación automática	Sandbox Docker · Tests unitarios · Scoring automático	PostgreSQL	REST / Queue
ms-exercises	Gestión de ejercicios	Labs web · Enunciados HTML · Plantillas · Dificultad	PostgreSQL	REST
ms-metrics	Métricas colaboración	IPB · Tiempo edición · Uso IA · Kafka Streams	InfluxDB	Events (Kafka)
ms-notifications	Notificaciones web	Push browser · Email · WebSocket · SMTP	Redis	WebSocket / SMTP
ms-reports	Reportes académicos	CQRS: QueryStack/CommandStack · Dashboard · PDF · S3	PostgreSQL + S3	REST

3.1 Descripción de capas

Capa 1 — Clientes Web (React.js SPA)

- React.js — Vista Estudiante: Editor de código web (Monaco Editor/CodeMirror), panel de sugerencias IA, chat de sesión y estado del par (driver/navigator).
- React.js — Vista Docente: Dashboard de supervisión en tiempo real, creación de laboratorios, visualización de métricas de participación.
- React.js — Vista Administrador: Gestión institucional, configuración de la plataforma, reportes globales.

Capa 2 — API Gateway (Kong)

- Punto de entrada único para todos los clientes web. Valida JWT, aplica SSO, rate limiting (100 req/min), CORS, routing y headers de trazabilidad X-Trace-ID.

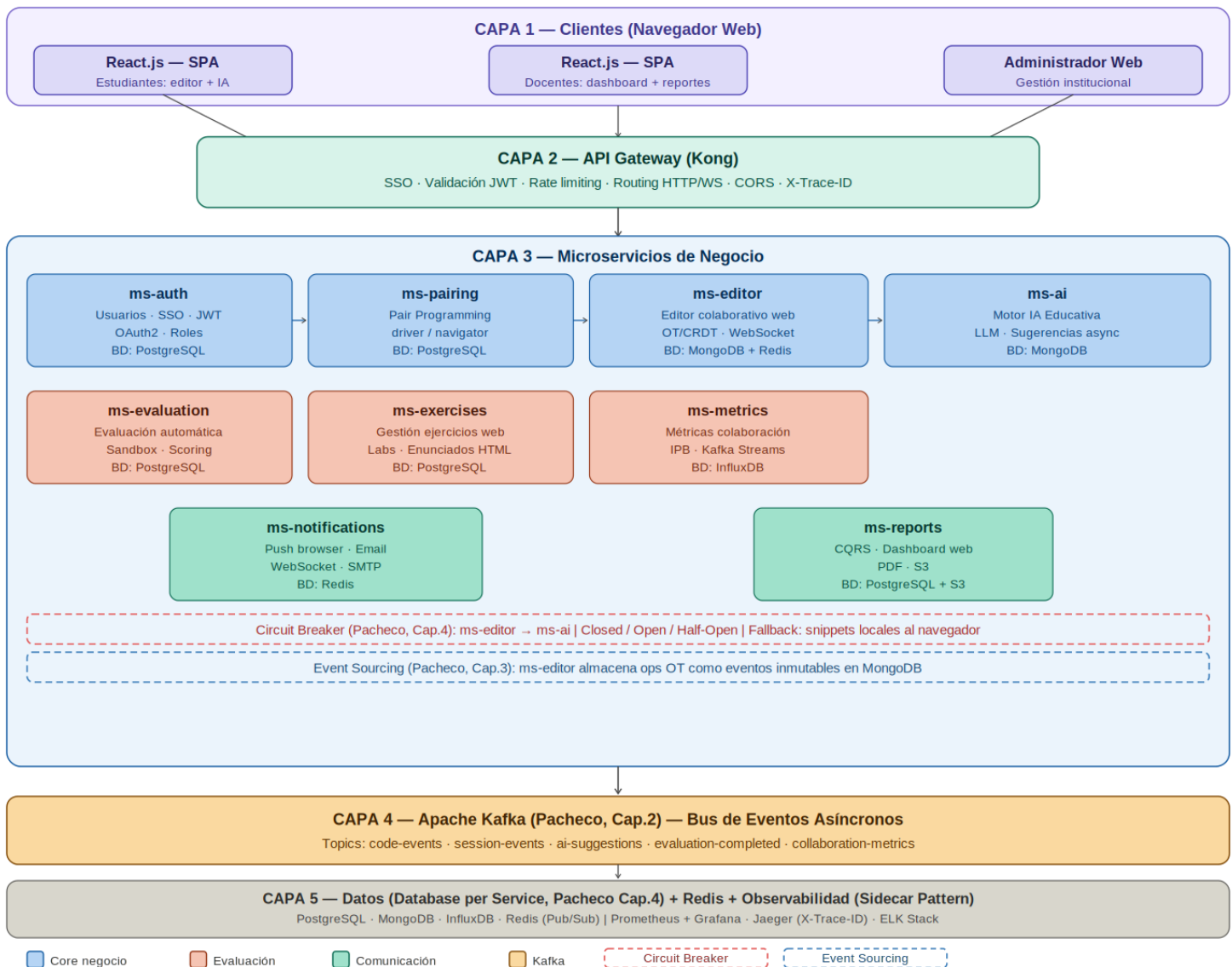
Capa 3 — Microservicios de Negocio

- Nueve microservicios independientes, cada uno con su propia base de datos (Database per Service). Comunicación sincrónica vía REST/WebSocket y asíncrona vía Apache Kafka.

Capas 4 y 5 — Infraestructura

- Apache Kafka como bus de eventos asíncronos. Redis para caché, Pub/Sub de WebSockets y OT buffer. Bases de datos independientes por servicio. Observabilidad con Prometheus, Grafana, Jaeger y ELK Stack usando el patrón Sidecar.

3.2 Diagrama de bloques



4. Diseño de Arquitectura Completa Cliente Web-Microservicio

4.1 Patrones de diseño aplicados

SSO — Single Sign-On (Pacheco, Cap. 13)

El ms-auth implementa SSO mediante OAuth2 + OpenID Connect con credenciales institucionales UNAB. El token JWT (RS256, vigencia 15 min) se almacena en memoria del SPA React.js para evitar ataques XSS. El refresh token (7 días) se guarda en cookie HTTPOnly para proteger contra CSRF. Los claims incluyen: userId, role, sessionId, institutionId.

API Gateway — Kong (Pacheco, Cap. 13)

Kong centraliza el punto de entrada de todos los clientes web. Funciones principales: routing por path (/auth, /sessions, /editor, /ai, /reports), validación JWT en cada request, rate limiting (100 req/min REST, sin límite para WebSocket), SSL termination, política CORS para el dominio del SPA, y headers de trazabilidad X-Trace-ID. Sin el gateway, la SPA React.js necesitaría conocer la URL de cada microservicio directamente (Pacheco, Cap. 13).

CQRS — ms-reports (Pacheco, Cap. 3 y 6)

El ms-reports implementa CQRS separando el CommandStack (escritura de métricas desde Kafka) del QueryStack (lectura para dashboards web del docente). Pacheco (Cap. 12) explica: si el Chaos Monkey apaga el servidor del CommandStack, el QueryStack de lectura continúa operando, garantizando que los docentes vean sus dashboards aunque no se estén registrando nuevas métricas.

Event Sourcing — ms-editor (Pacheco, Cap. 3)

El ms-editor almacena todas las operaciones OT como eventos inmutables en MongoDB. Esto permite reconstruir el estado del documento en cualquier momento y auditar la historia completa de edición de cada pareja, lo cual es fundamental para la evaluación académica.

4.2 Flujos de comunicación principales

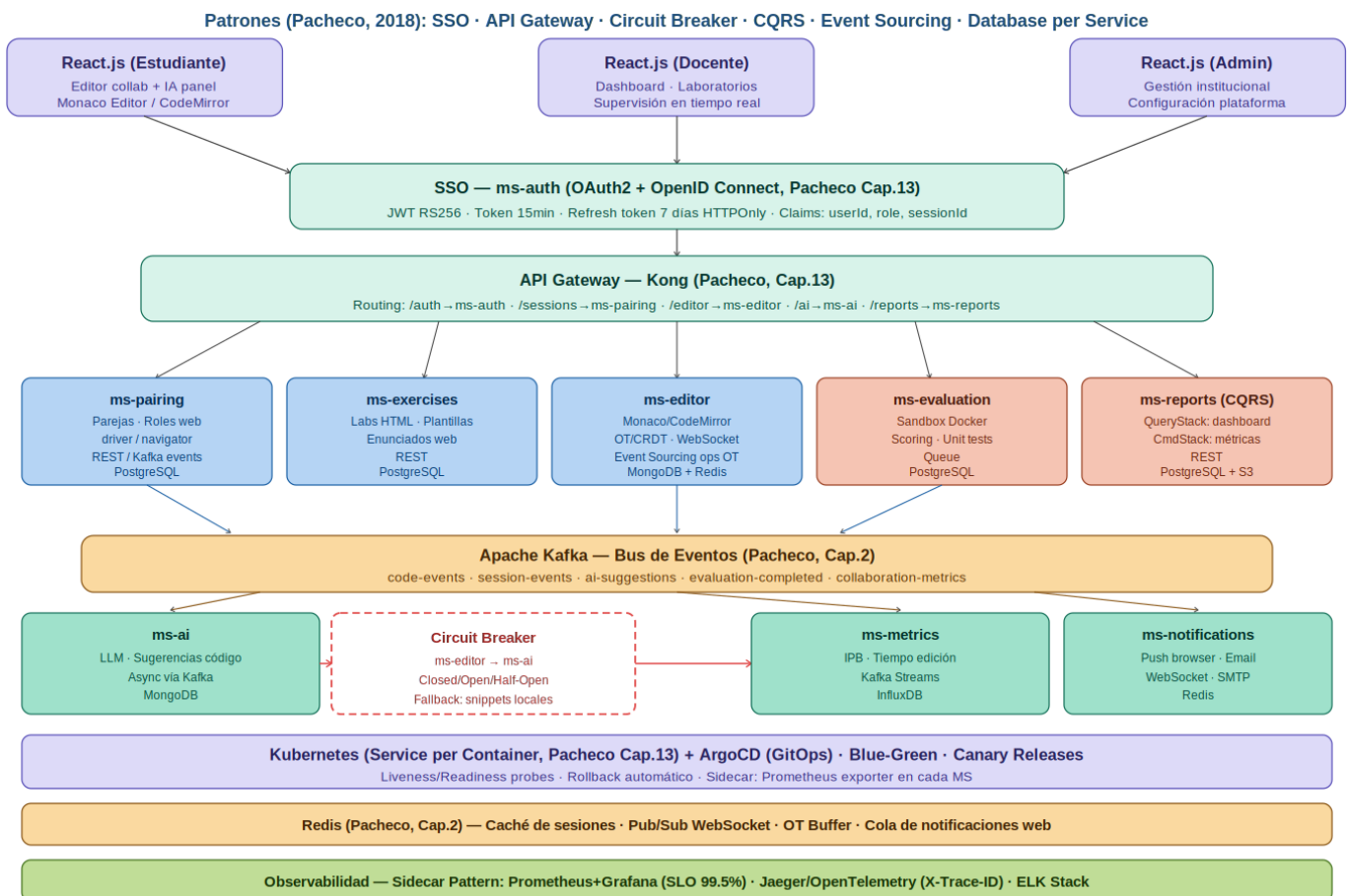
Flujo 1: Autenticación y creación de sesión web

- 1) Estudiante A abre la SPA en el navegador. React Router carga el componente de login.
- 2) POST /api/auth/login → ms-auth devuelve JWT. El SPA almacena el JWT en memoria (no en localStorage, para proteger contra XSS).
- 3) Estudiante A solicita sesión de Pair Programming: POST /api/sessions/create → ms-pairing crea la sesión.
- 4) ms-pairing emite evento a Kafka y ms-notifications envía una notificación push al navegador del Estudiante B.
- 5) Estudiante B acepta → ms-pairing confirma pareja y ambos son redirigidos al editor colaborativo web.

Flujo 2: Edición colaborativa con sugerencias IA en el navegador

- 1) Estudiante A (driver) escribe en Monaco Editor del navegador. La operación OT se envía vía WebSocket al ms-editor.
- 2) ms-editor aplica transformación, actualiza Redis y retransmite la operación al navegador del Estudiante B (navigator). El cursor de A aparece en tiempo real en el editor de B.
- 3) Cada 10 segundos, ms-editor publica un snapshot del código al topic 'code-events' de Kafka.
- 4) ms-ai consume el snapshot, analiza con el modelo LLM y publica sugerencias al topic 'ai-suggestions'. Circuit Breaker activo.
- 5) ms-editor consume la sugerencia y la envía vía WebSocket al navegador. El panel de IA del editor web muestra la sugerencia para ambos estudiantes.
- 6) ms-metrics registra el evento en InfluxDB. Si el Circuit Breaker está abierto (ms-ai no disponible), el panel web muestra 'Asistencia IA temporalmente no disponible' y carga snippets locales.

4.3 Diagrama de arquitectura completa



5. Pipeline de Integración y Despliegue Continuo (CI/CD)

5.1 Descripción (Pacheco, Cap. 12 y 13)

Pacheco (Cap. 12) define las etapas de pruebas que deben preceder al despliegue: unit tests (pruebas aisladas con mocks), integration tests (con BD real), end-to-end tests (flujo completo de negocio), signature tests (contratos entre servicios) y monkey tests (caos controlado). El Cap. 13 describe el pipeline completo de CI/CD con los patrones Blue-Green y Canary Releases, y el despliegue Service per Container en Kubernetes.

Para la plataforma web, el Build incluye el proceso de Webpack/Vite para compilar la SPA React.js en un artefacto dist/ servido por Nginx, y los E2E tests usan Cypress (herramienta diseñada para pruebas web) en lugar de Postman.

Etapas	Herramienta	Acción	Criterio de calidad
1. Source Control	Git / GitHub	Push feature branch → PR → revisión → merge develop	Protección de ramas obligatoria
2. Build	Docker + Webpack/Vite	Compilación React · imagen Docker versionada (semver)	Artefacto: dist/ → imagen Nginx
3. Análisis estático	SonarQube	Calidad código · vulnerabilidades · deuda técnica	Quality Gate: cobertura >80% · 0 bugs críticos
4. Unit Tests *	Jest / JUnit	Pruebas unitarias con mocks · determinísticas por MS	Falla pipeline si cobertura <80%
5. Integration Tests *	Testcontainers	BD real en contenedor · contratos entre microservicios	Verifica flujos de integración reales
6. Signature Tests *	git diff + CI	Verifica contratos API OpenAPI · cambios en interfaces	Alerta a equipos si firma cambia
7. Container Registry	Docker Hub / ECR	Push imagen con SHA digest inmutable al registro	Inmutabilidad garantizada por digest
8. Deploy Staging	Kubernetes + Helm	Réplica de producción · health checks · readiness probes	Entorno idéntico a producción
9. E2E Tests *	Cypress (web)	Flujo completo Pair Programming en staging	Smoke tests sobre editor y IA web

Etapas	Herramienta	Acción	Criterio de calidad
10. Monkey Tests *	Chaos Mesh	Caos controlado en staging · Circuit Breaker verificado	Fallback IA comprobado en navegador
11. Deploy Producción	ArgoCD (GitOps)	Blue-Green sin downtime · Canary ms-ai 5%→100%	Rollback automático si health check falla
12. Monitoreo	Prometheus+Grafana	Métricas · Jaeger (trazas) · ELK (logs)	SLO: disponibilidad >99.5%

5.2 Estrategias de despliegue (Pacheco, Cap. 13)

Blue-Green Deployment

Se mantienen dos entornos idénticos (Blue = activo, Green = nuevo). Al completar el pipeline, el API Gateway conmuta el tráfico de Blue a Green en segundos sin downtime. Si Green falla los health checks, el rollback es inmediato volviendo a Blue. El CDN (CloudFront) distribuye los assets estáticos de la SPA React.js para reducir latencia en el navegador.

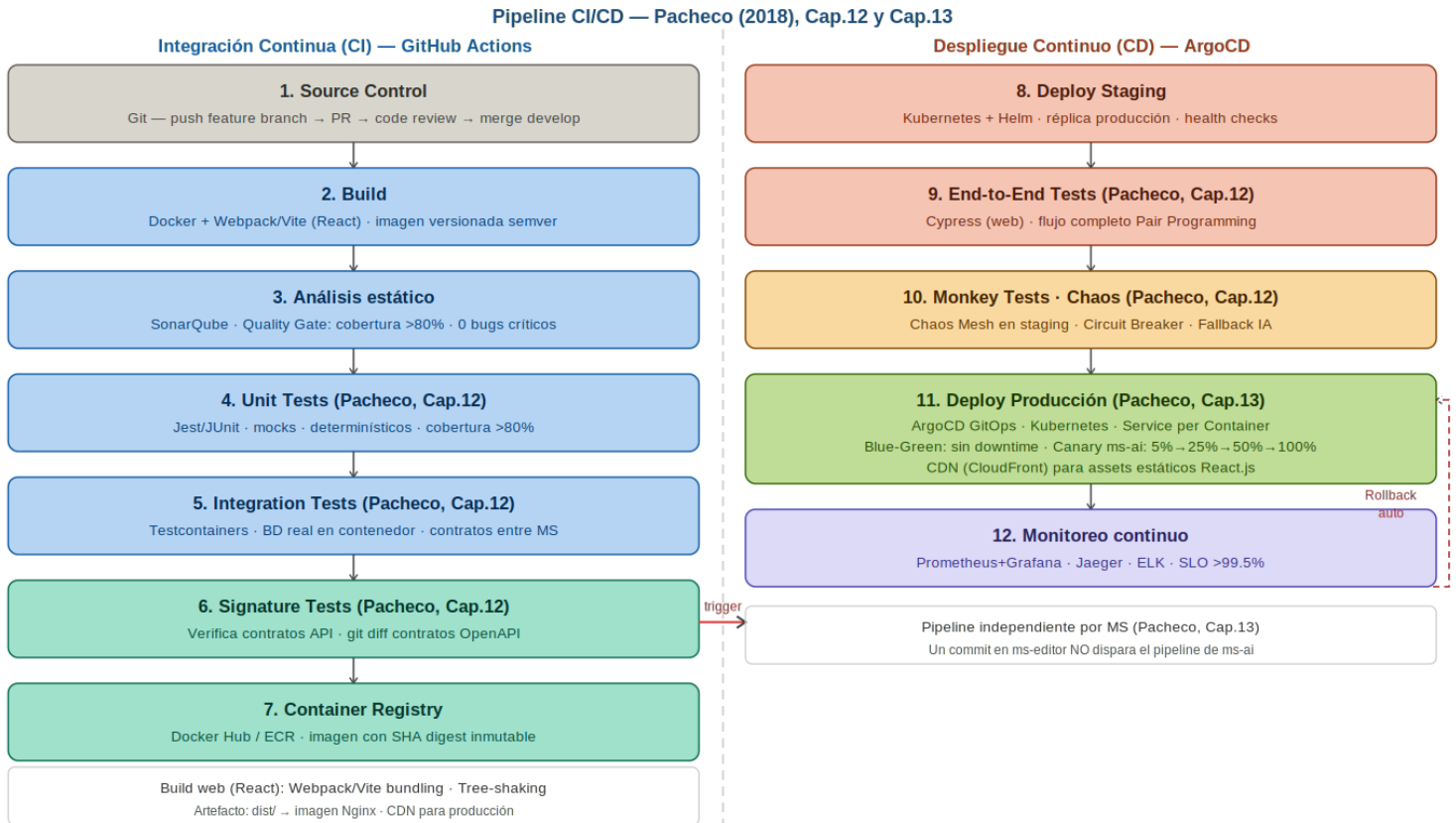
Canary Releases (Pacheco, Cap. 13)

Para el ms-ai, se dirige el 5% del tráfico al nuevo modelo LLM durante 30 minutos. Si las métricas (latencia, error rate, calidad de sugerencias) son correctas, se incrementa a 25%, 50% y 100%. Esto minimiza el riesgo de que un modelo de IA con comportamiento inesperado afecte a todos los estudiantes simultáneamente.

Service per Container (Pacheco, Cap. 13)

Cada microservicio corre en su propio contenedor Docker gestionado por Kubernetes. La SPA React.js se sirve desde una imagen Nginx ligera. Kubernetes gestiona liveness probes (¿el servicio está vivo?) y readiness probes (¿puede recibir tráfico?).

5.3 Diagrama del pipeline CI/CD



6. Prueba del Mono (Chaos Engineering)

6.1 Introducción (Pacheco, Cap. 12)

Pacheco (Cap. 12) define las Monkey Tests como pruebas automatizadas con valores aleatorios que buscan errores en la aplicación. El Chaos Monkey, creado por Netflix, termina servidores aleatoriamente en producción durante horas de negocio para medir la resiliencia. El principio fundamental es: si el sistema no sobrevive a una falla controlada en pruebas, no sobrevivirá a una falla real en producción.

Pacheco describe explícitamente el comportamiento de CQRS bajo Chaos Monkey (Cap. 12): si el Chaos Monkey apaga el servidor de la base de datos del CommandStack (escritura), el proceso de lectura del QueryStack no se interrumpe. Los dashboards web de los docentes siguen funcionando aunque no se registren nuevas métricas. Si la cola es transaccional, la información vuelve a la cola hasta que la instancia se restaure.

Las siete categorías del Simian Army de Netflix aplicadas a esta plataforma web son:

Mono	Servicios	Falla inyectada	Comportamiento esperado	Severidad
Chaos Monkey	ms-pairing, ms-auth	Termina instancias aleatorias en producción	K8s reinicia en <30s · sesiones web recuperadas desde Redis	Crítico
Latency Monkey	ms-editor, ms-ai	Inyecta 500ms–3s de latencia artificial	Circuit Breaker activa · editor continúa · IA deshabilita con aviso visual	Alto
Security Monkey	ms-auth, Gateway	XSS · CSRF · JWT inválido · SQL injection	Gateway bloquea · CSP/HSTS activos · logs de seguridad generados	Crítico
Conformity Monkey	API Gateway	Verifica adherencia a mejores prácticas y contratos OpenAPI	MS que no cumplen se desconectan del gateway automáticamente	Alto
Doctor Monkey	Todos los MS	Health checks periódicos de CPU y memoria	Prometheus alerta · Grafana notifica · MS sale del load balancer	Medio
Janitor Monkey	ms-metrics, ms-reports	Elimina cachés Redis y recursos sin uso	MS reconstruye estado desde BD · sin pérdida de datos persistentes	Medio
Chaos Gorilla	Zona completa	Simula caída de toda una zona de disponibilidad cloud	K8s redistribuye en otras zonas · multi-región garantiza continuidad	Alto
Monkey/Fuzzing	ms-notifications, ms-reports	Valores aleatorios inválidos (fuzzing de entradas web)	UTF-8 correcto · sin corrupción de datos · logs bien codificados	Bajo

6.2 Consideraciones especiales para plataforma web

- Security Monkey agrega verificación de headers de seguridad web específicos: Content-Security-Policy (CSP), HTTP Strict Transport Security (HSTS) y X-Frame-Options, fundamentales para proteger la SPA React.js de ataques XSS y clickjacking.
- Latency Monkey verifica que el editor web degrade elegantemente: el Circuit Breaker activa el fallback con snippets locales en el navegador, sin mostrar pantallas de error técnico al estudiante.
- Chaos Gorilla garantiza que el CDN siga sirviendo los assets estáticos de la SPA React.js aunque toda una zona de disponibilidad cloud caiga.

6.3 Criterios de éxito post-caos

- Degradación elegante en el navegador: ante falla de ms-ai, el editor web continúa sin IA con aviso visual claro al estudiante (Circuit Breaker, Pacheco Cap. 4).
- CQRS resiliente (Pacheco, Cap. 12): si cae el CommandStack, los dashboards web del docente (QueryStack) siguen operando.
- Recuperación automática: Kubernetes reinicia pods en menos de 30 segundos. Las sesiones web se recuperan desde Redis.
- Sin pérdida de datos: Kafka retiene eventos 7 días. Ningún snapshot de código o métrica se pierde durante fallos transitorios.
- Trazabilidad: cada experimento de caos queda registrado en Jaeger con X-Trace-ID y en ELK Stack para análisis forense completo.

6.4 Diagrama de la Prueba del Mono

Prueba del Mono — Chaos Engineering (Pacheco, 2018, Cap.12)

Sistema en estado estable

Plataforma Web Pair Programming IA — UNAB

Categorías: Chaos Monkey - Chaos Gorilla - Conformity Monkey - Doctor Monkey - Janitor Monkey - Latency Monkey - Security Monkey

Chaos Monkey (Pacheco, Cap.12)

CRÍTICO

Servicios: ms-pairing, ms-auth

Falla: termina instancias aleatorias en producción

Respuesta: Kubernetes reinicia pods en menos de 30s

Sesiones web recuperadas desde Redis · reconexión auto

Latency Monkey (Pacheco, Cap.12)

ALTO

Servicios: ms-editor, ms-ai

Falla: inyecta 500ms – 3s de latencia artificial

Respuesta: Circuit Breaker activa fallback al navegador

Editor web continúa · IA deshabilita con aviso visual

Security Monkey (Pacheco, Cap.12)

CRÍTICO

Servicios: ms-auth, API Gateway

Falla: XSS, CSRF, JWT inválido, SQL injection

Respuesta: Gateway bloquea · logs generados

Headers de seguridad web: CSP, HSTS, X-Frame-Options

Conformity Monkey (Pacheco, Cap.12)

ALTO

Servicio: API Gateway

Falla: verifica cumplimiento de mejores prácticas

Respuesta: MS que no cumplen contrato OpenAPI se desconectan del gateway automáticamente

Doctor Monkey (Pacheco, Cap.12)

MEDIO

Servicios: todos los microservicios

Falla: health checks periódicos de CPU y memoria

Respuesta: Prometheus detecta · Grafana alerta

MS degradado sale del load balancer de Kubernetes

Janitor Monkey (Pacheco, Cap.12)

MEDIO

Servicios: ms-metrics, ms-reports

Falla: elimina cachés Redis y recursos sin uso

Respuesta: MS reconstruye estado desde BD

Sin pérdida de datos persistentes

Chaos Gorilla (Pacheco, Cap.12)

ALTO

Alcance: zona de disponibilidad completa

Falla: simula caída de toda una zona cloud

Respuesta: K8s redistribuye en otras zonas

Multi-región garantiza continuidad del servicio web

Monkey Tests / Fuzzing (Pacheco, Cap.12)

BAJO

Servicios: ms-notifications, ms-reports

Falla: valores aleatorios inválidos (fuzzing)

Respuesta: UTF-8 correcto · sin corrupción

Logs bien codificados en ELK Stack

CQRS bajo Chaos Monkey (Pacheco, Cap.12):

Si el Chaos Monkey apaga el CommandStack de ms-reports, el QueryStack (dashboards web del docente) continúa operando sin interrupción

Herramientas de caos

Chaos Mesh (Pod/Red/Disco en K8s) · Toxiproxy (latencia/paquetes) · k6 (carga: 500+ sesiones web) · Gremlin (CPU/RAM)

Criterios de éxito post-caos (Pacheco, Cap.12)

Degradación elegante: falla ms-ai → Circuit Breaker → editor web continúa sin IA con aviso visual al usuario

CQRS resiliente: CommandStack cae → QueryStack (dashboards) sigue operando para docentes

Recuperación automática: K8s reinicia pods en menos de 30s · Kafka retiene 7 días sin pérdida de datos

Trazabilidad: cada experimento registrado en Jaeger con X-Trace-ID · logs en ELK Stack

● Crítico ● Alto ● Medio ● Bajo

Categorías según Pacheco, Microservice Patterns and Best Practices, Cap.12

7. Conclusiones

El diseño de arquitectura de microservicios presentado constituye una base sólida, escalable y resiliente para la Plataforma Web Inteligente de Pair Programming de la UNAB. Aplicando los patrones del libro de Pacheco (2018) como referencia, las principales conclusiones son:

- La separación en nueve microservicios con Database per Service (Pacheco, Cap. 4) garantiza independencia funcional, tecnológica y de datos, permitiendo que cada equipo despliegue su servicio de forma autónoma.
- El desacoplamiento asíncrono vía Kafka entre ms-editor y ms-ai (Asynchronous Messaging, Pacheco Cap. 10) es el elemento clave que garantiza una experiencia fluida en el navegador, independientemente del estado del motor de IA.

- El Circuit Breaker (Pacheco, Cap. 4) con degradación elegante asegura que los estudiantes sigan trabajando en el editor web aunque el ms-ai no esté disponible.
- CQRS en ms-reports (Pacheco, Cap. 3 y 6) garantiza que los dashboards web de los docentes sigan operando aunque el CommandStack falle, como explica Pacheco en el Cap. 12.
- El pipeline CI/CD con las etapas de Pacheco (Cap. 12 y 13), incluyendo Cypress para E2E tests web y Chaos Mesh para Monkey Tests, permite entregas frecuentes y seguras sobre la plataforma web.
- La Prueba del Mono con las siete categorías del Simian Army (Pacheco, Cap. 12) garantiza que la plataforma web resiste fallas reales antes de que estas ocurran en producción, protegiendo la continuidad académica de los estudiantes de la UNAB.